Backup, Recovery & Reset

Kubernetes Production · Lesson 18

The manual etcd skill (CKA) + Kubespray automation

`kubernetes.io/docs` · `kubespray.io`

Learning Objectives

By the end of this lesson you will be able to:

- Explain why **etcd is the source of truth** and why a **snapshot is your backup**.
- Take an etcd **snapshot** with `etcdctl` (v3 API + TLS certs) and verify it with `snapshot status`.
- **Restore** a snapshot into a **new data directory**, moving static-pod manifests aside to stop etcd/apiserver during the restore.
- Recognize Kubespray's `host` etcd deployment (certs in `/etc/ssl/etcd/ssl/`, client `2379`, metrics `2381`).
- Recover a broken control plane with `recover-control-plane.yml` and tear a cluster down with `reset.yml`.
- Enable **certificate auto-renewal** and **encryption of Secrets at rest**.

18.1 Why Back Up etcd?

etcd is the Source of Truth

- **etcd is the backing store for ALL cluster data** — Deployments, Secrets, ConfigMaps, RBAC, every object the API server serves.
- Access to etcd is **equivalent to root on the cluster**. Lose etcd → **lose the cluster state**.
- The backup primitive is a **point-in-time snapshot**; the restore primitive writes it into a **fresh, empty data directory** as a new etcd member.
- Two layers we cover:
  - **Manual** `etcdctl` — the universal, CKA-exam skill (any kubeadm/host cluster).
  - **Kubespray playbooks** — automate the same workflow at cluster scale.

> Treat snapshots as **disaster-recovery backups**: take them on a schedule and **copy them off the node**.

18.2 Manual etcd Backup with etcdctl

Taking a Snapshot

Use `etcdctl` with the **v3 API** and TLS client certs. On a **kubeadm** cluster the certs are under `/etc/kubernetes/pki/etcd/`.

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/snapshot.db
```

Flag	Meaning
`--endpoints`	etcd client endpoint(s) — client port **2379**
`--cacert`	CA cert to **verify the server**
`--cert` / `--key`	**client** certificate and key (mutual TLS)

Verifying the Snapshot

```
ETCDCTL_API=3 etcdctl snapshot status /opt/snapshot.db --write-out=table
```

- Reports **hash**, **revision**, **total keys**, and **db size** — confirming the snapshot is **readable** and not obviously truncated.
- **Important:** `snapshot status` is a **sanity check, not a full integrity guarantee**. It shows the file parses and prints its metadata — it does **not** prove the backup will restore cleanly.
- Still, **always run it right after a save** — it catches the obvious failures.

18.3 Manual etcd Restore

Restore Prerequisites

1. **Stop all etcd members** and **all kube-apiserver instances** for the duration of the restore.
2. Restore must go into a **NEW, empty data directory** — **never** overwrite the live one in place.

> **Move static-pod manifests aside.** On a static-pod control plane, moving `etcd.yaml` (and `kube-apiserver.yaml`) **out** of `/etc/kubernetes/manifests/` stops those static Pods cleanly; moving them **back** restarts them. That is how you bounce etcd/apiserver around a restore.

Restore to a New Data Dir

```
ETCDCTL_API=3 etcdctl snapshot restore /opt/snapshot.db \
  --data-dir=/var/lib/etcd-restore
```

- Writes the snapshot's data into the **new** directory as a **fresh etcd member**.
- The live data directory is left **untouched** — if something goes wrong you can start over from the snapshot.

Point etcd at the Restored Data

1. Edit the etcd **static Pod manifest** `/etc/kubernetes/manifests/etcd.yaml`.
2. Update the `hostPath` volume for the etcd data dir to `/var/lib/etcd-restore` (and the `--data-dir` flag to match).
3. Move the manifests **back** → the **kubelet recreates** etcd against the restored data.
4. **Restart the kube-apiservers** and confirm cluster state with `kubectl get nodes` / `kubectl get pods -A`.

18.4 Kubespray etcd Deployment

Kubespray Runs etcd on the Host

- Kubespray's default is `etcd_deployment_type: host` — etcd runs as a **systemd service** on the `etcd` group nodes.
  - Other options: `kubeadm` (static pod) or `docker`.
- Client TLS material lives under `/etc/ssl/etcd/ssl/` — **not** `/etc/kubernetes/pki/etcd/`.
- Client port `2379`; metrics on `etcd_metrics_port: 2381`. Backups use the **client** port.

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd_snapshot \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/ssl/etcd/ssl/ca.pem \
  --cert=/etc/ssl/etcd/ssl/member-<node>.pem \
  --key=/etc/ssl/etcd/ssl/member-<node>-key.pem
```

18.5 Recovering a Broken Control Plane

Recovery Runbook

Use `recover-control-plane.yml` for hardware failure, failure during patch/upgrade, **etcd corruption**, or any degraded control plane.

> **Prerequisite:** you need **at least one functional node** to recover.

- Back up what you can; provision new nodes for the broken ones.
- Put broken etcd nodes in the `broken_etcd` group (set `etcd_member_name`).
- Put broken control-plane nodes in `broken_kube_control_plane`.
- List the **surviving** nodes **first** in the `etcd` / `kube_control_plane` groups; add the **new** nodes **below** them.

Run recover-control-plane.yml

Scope the run to the control-plane/etcd groups and raise the etcd retry count (the right value is hard to predict — go larger if needed):

```
ansible-playbook -i inventory/mycluster/hosts.yaml \
  recover-control-plane.yml \
  --limit etcd,kube_control_plane \
  -e etcd_retries=10
```

- `--limit etcd,kube_control_plane` targets exactly those groups — the broken member plus a healthy node, so the playbook rebuilds from a working quorum.

Recover from Lost Quorum

The playbook **detects whether etcd quorum is intact**. If quorum is **lost**, it takes a snapshot from the **first node in the `etcd` group** and restores from it. Pass a path to restore a **specific** snapshot:

```
ansible-playbook -i inventory/mycluster/hosts.yaml \
  recover-control-plane.yml \
  --limit etcd,kube_control_plane \
  -e etcd_retries=10 \
  -e etcd_snapshot=/tmp/etcd_snapshot
```

> **Caveats (docs):** only tested with **small etcd DBs**; **may cause disruptions**; **no guarantees**. Practice on a test cluster first.

18.6 Tearing Down with reset.yml

reset.yml — Destructive Inverse of cluster.yml

`reset.yml` removes Kubernetes from the target nodes — the **inverse of `cluster.yml`**. Useful for redeploying from scratch or decommissioning.

```
ansible-playbook -i inventory/mycluster/hosts.yaml reset.yml \
  -e reset_confirmation=yes
```

- It is **destructive**, so Kubespray **requires an explicit confirmation**.
- `-e reset_confirmation=yes` skips the interactive prompt — only when you are certain.

> It wipes K8s components, **etcd data**, CNI and config on the nodes. **Back up etcd first.**

18.7 & 18.8 Certificates & Encryption at Rest

Certificate Auto-Renewal

kubeadm issues **1-year** control-plane certs by default; Kubespray can **auto-renew** them so they never lapse silently.

- `auto_renew_certificates: false` (default) → set `true` to auto-renew on a schedule (default: **first Monday of each month**).
- Configurable via `auto_renew_certificates_systemd_calendar`, e.g. `"Mon *-*-1,2,3,4,5,6,7 03:00:00"` — installs a **systemd timer/service** on the control plane.
- Re-running `cluster.yml` / `upgrade-cluster.yml` also refreshes certs via the kubeadm flow; auto-renewal is the hands-off option between runs.

> Set `auto_renew_certificates: true` on long-lived clusters so an expired apiserver/etcd cert never takes the control plane down.

Encrypting Secret Data at Rest

By default Secrets are stored **base64-encoded, not encrypted**, in etcd.
Kubespray enables encryption via a kube-apiserver `EncryptionConfiguration`.

- **`kube_encrypt_secret_data: false`** (default) → `true` generates a key and wires an **`EncryptionConfiguration`** into the apiserver.
- **5 providers:** `identity`, `aesgcm`, `aescbc`, **`secretbox`** (default), `kms`.

Provider	Why / why not (default = **`secretbox`**)
`identity`	no encryption
`aesgcm`	must be **rotated every 200k writes**
`aescbc`	**not recommended** (padding-oracle risk)
`secretbox`	**default** — Poly1305 MAC + XSalsa20
`kms`	officially recommended, but needs an external KMS

> Enabling it only encrypts **newly written** Secrets until you re-write existing ones.

Key Takeaways

- **etcd is the source of truth → a snapshot is your backup.** Take them on a schedule, **copy off-node**, verify with `snapshot status` (a sanity check).
- **Manual backup:** `ETCDCTL_API=3 etcdctl snapshot save` with the four flags (`--endpoints --cacert --cert --key`); kubeadm certs in `/etc/kubernetes/pki/etcd/`.
- **Manual restore:** new `--data-dir`, **move static-pod manifests aside**, then repoint the etcd manifest at the restored dir.
- **Kubespray etcd is `host`** (systemd) — certs in `/etc/ssl/etcd/ssl/`, client `2379`, metrics `2381`.
- **`recover-control-plane.yml`** — `broken_etcd` / `broken_kube_control_plane` groups, `--limit etcd,kube_control_plane -e etcd_retries=…`, restore via `-e etcd_snapshot=…` on lost quorum.
- **`reset.yml -e reset_confirmation=yes`** is the destructive inverse of `cluster.yml`; `auto_renew_certificates` + `kube_encrypt_secret_data` are the preventive toggles.

End of Lesson 18

Next: Lesson 19 — Add-ons & Packaging

Cheat-sheet:

`etcdctl snapshot save /opt/snap.db --endpoints … --cacert … --cert … --key …` ·
`etcdctl snapshot status /opt/snap.db --write-out=table` ·
`etcdctl snapshot restore /opt/snap.db --data-dir=/var/lib/etcd-restore` ·
`recover-control-plane.yml --limit etcd,kube_control_plane -e etcd_retries=10 -e etcd_snapshot=…` ·
`reset.yml -e reset_confirmation=yes` ·
`auto_renew_certificates: true` · `kube_encrypt_secret_data: true` (`secretbox`)